CompSci 390-03-26S

# Distributed and Networked Systems

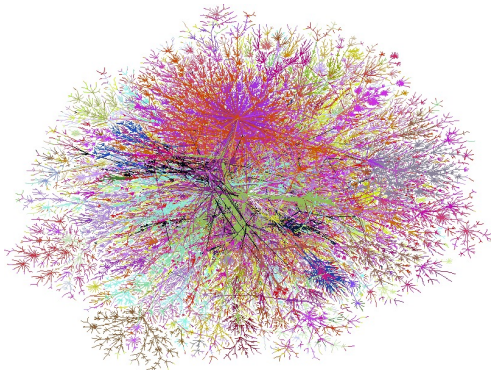
# Cryptosystem Primitives

# Path security

Anyone who owns a network on the transmit path can:

- intercept the packet
- transform/corrupt the bits
- block the packet or misdirect

"Internet treats censorship as failure and routes around it."

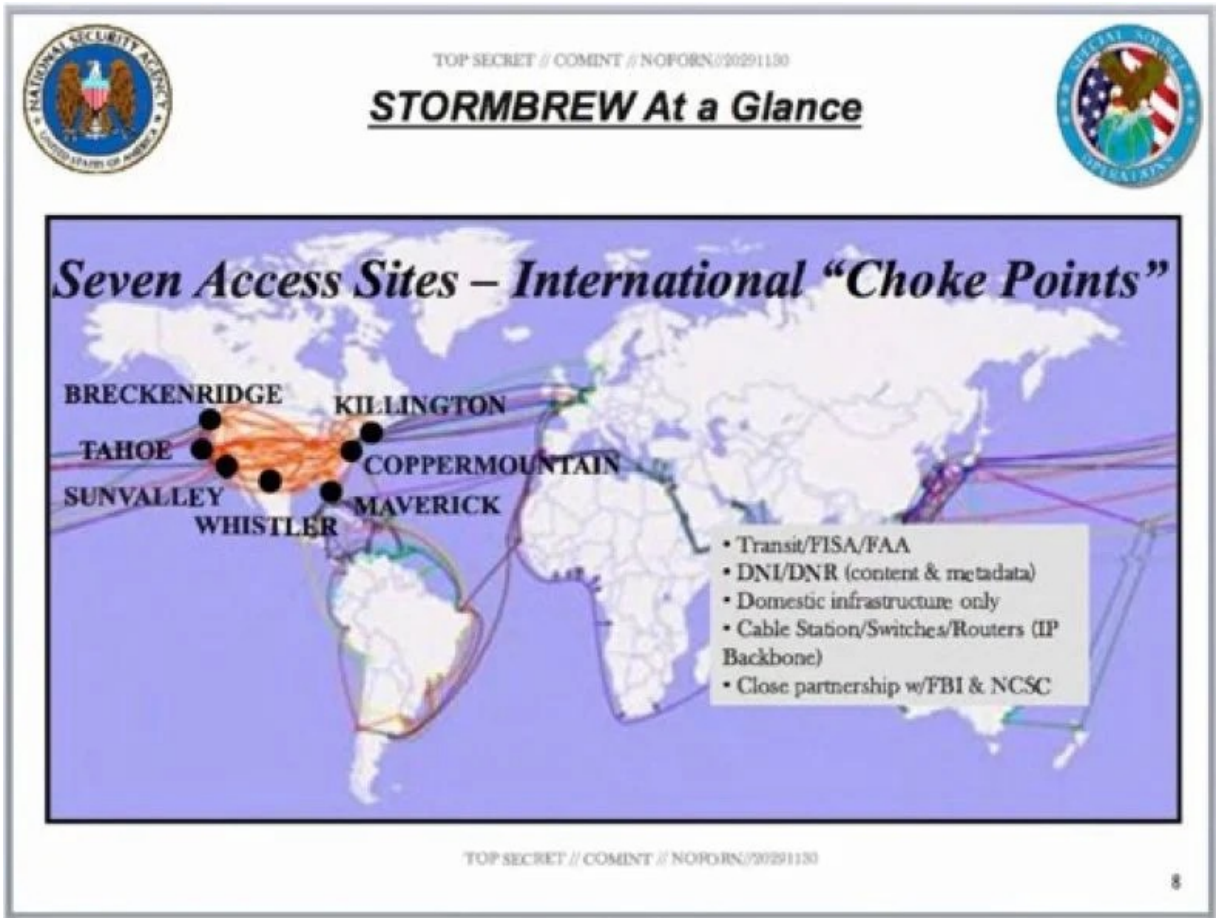


# Security properties

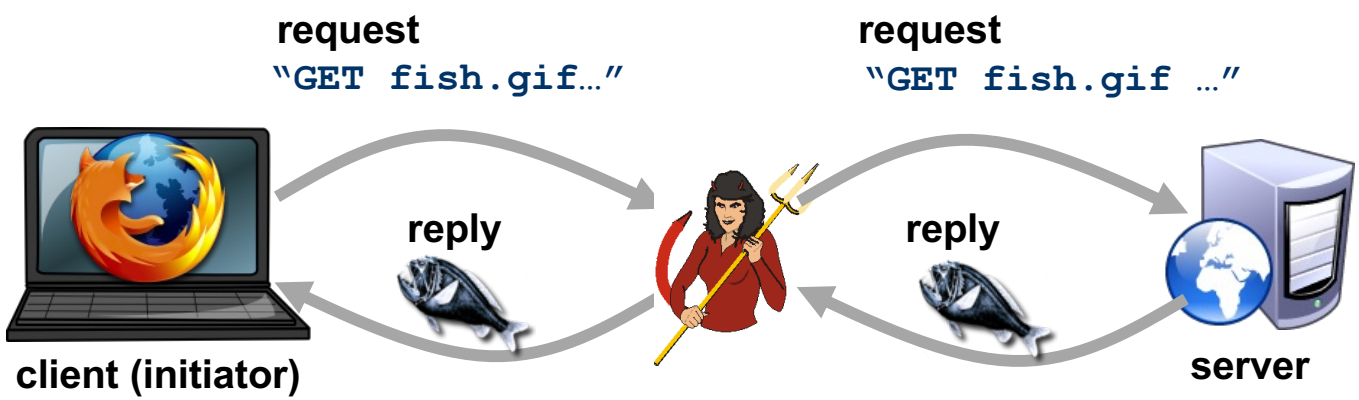
How to provide secure communication between Alice, Bob, Cindy...and Mal.

- **Authentication**: who sent the message?
- **Integrity**: is it what they intended to send?
- **Confidentiality**/privacy: no eavesdropping

# Stellar Wind and Stormbrew



# Mal-in-the-Middle attack

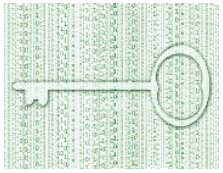



If an attacker can interpose between two interacting endpoints, she can read and modify the messages at will without detection by either victim. We need some crypto.

Classically a Man-in-the-Middle attack (MITM).

# Four crypto primitives

**Secure hashing**
useful for
fingerprinting data



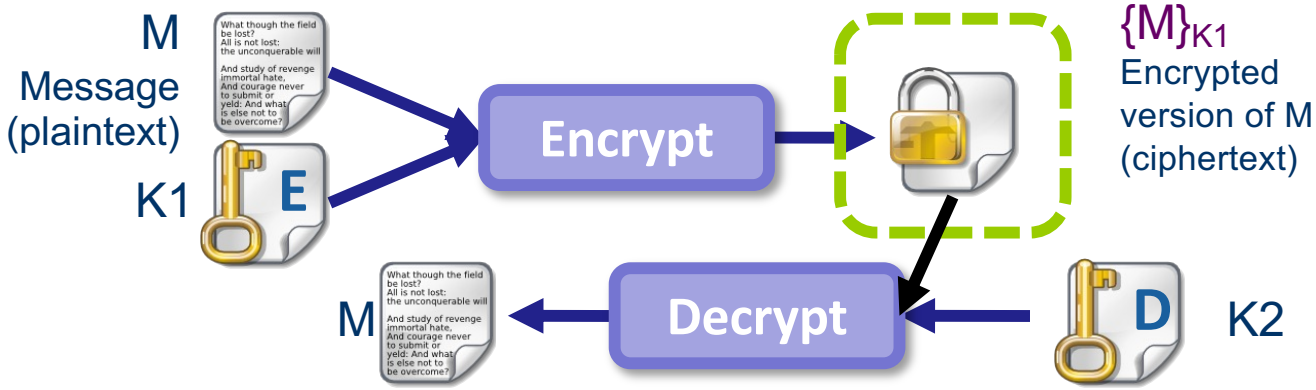
**Encrypting /Decrypting**

**Signing / Authenticating**

**Key exchange / agreement**

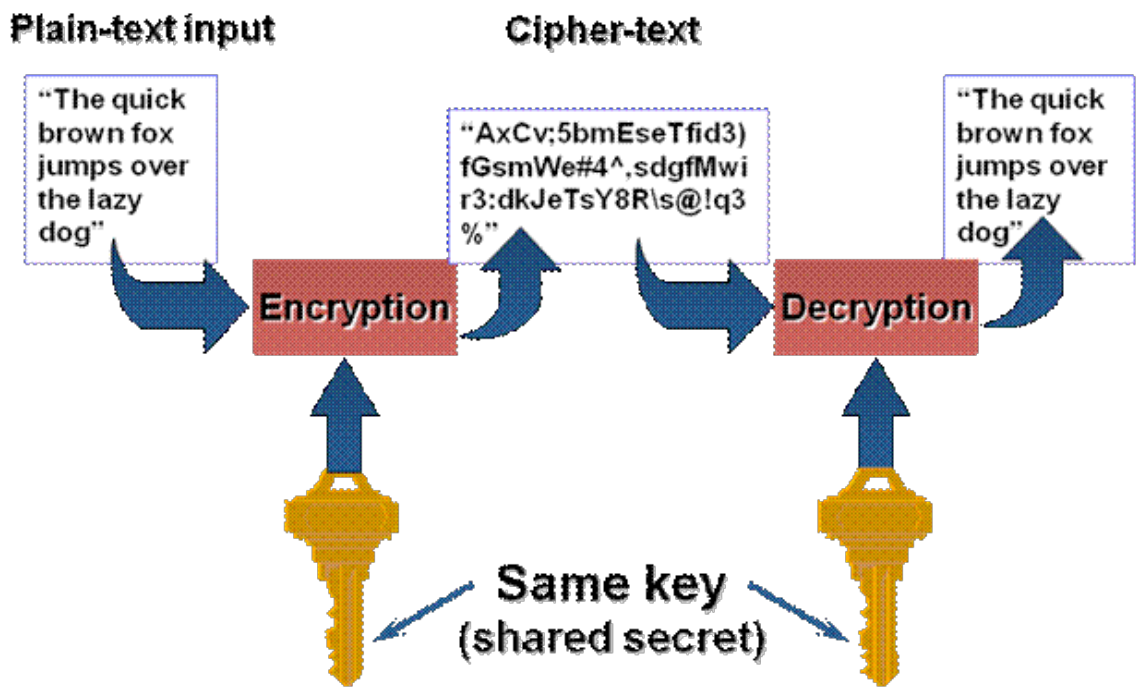
# Cryptography for Busy People

- Standard crypto functions parameterized by keys.
  - Key is a fixed-width "random" value (width matters!)
  - Symmetric (DES, AES): fast, requires shared key: K = K1 = K2
  - Asymmetric (classical: RSA): slow, uses two keys: a keypair {K1, K2}
- "Believed to be computationally infeasible" to break




[Image: Landon Cox]

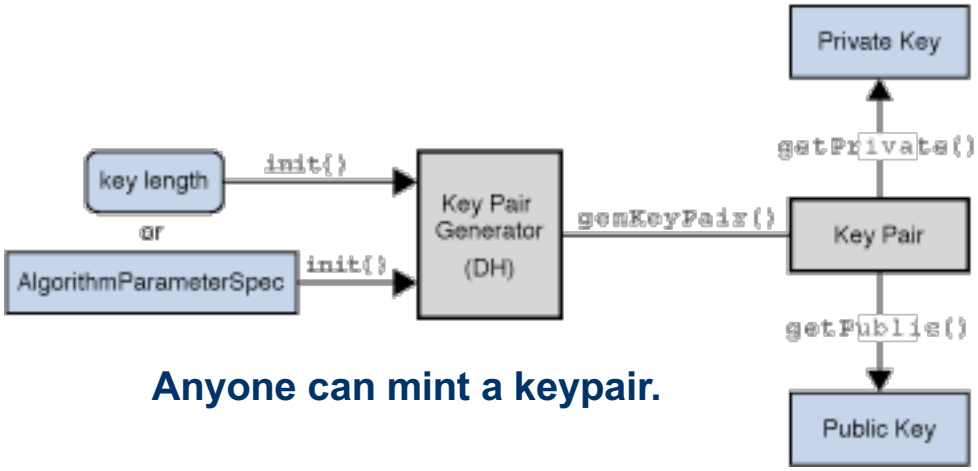
# Symmetric crypto




Borrowed from https://spideroak.com/blog/20130523083520-drink-your-ovaltine-intro-to-encryption-101

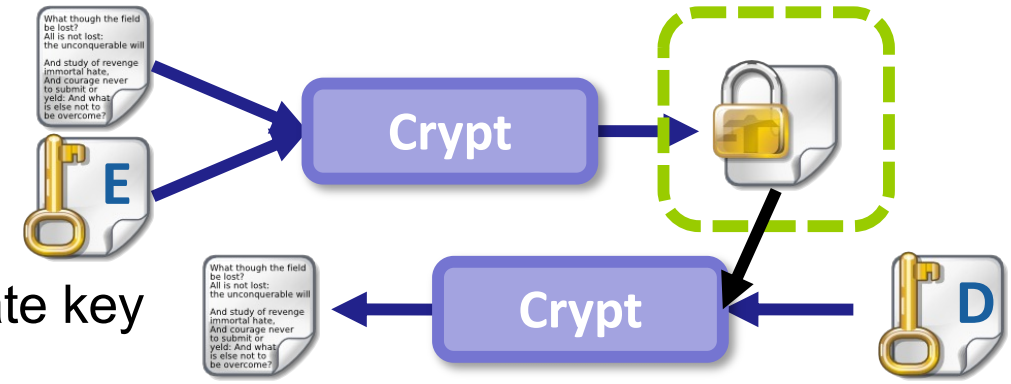
# Public key crypto (asymmetric)

- Each subject/principal possesses a keypair.
  - Decrypt(K, Encrypt($K^{-1}$, M)) = M
- Keep one key private; the other is public.
- Either key can be used to encrypt/decrypt.




**Anyone can mint a keypair.**

# Public key crypto works both ways




A's private key
or
A's public key

A's public key
or
A's private key

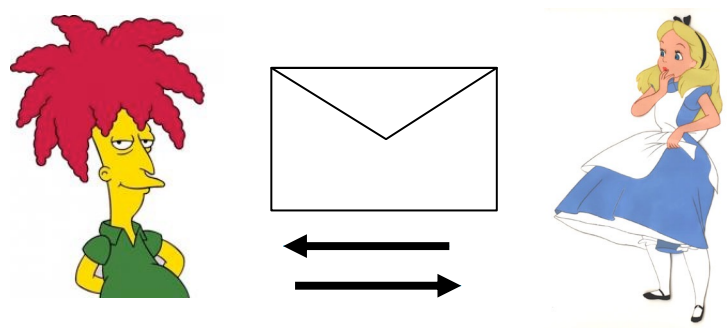
[Landon Cox]

# How to use public key crypto?

Consider:

- A sends a message to B, encrypted with A's **private** key.
- B sends a message to A, encrypted with A's **public** key.

Benefits? Other possibilities?

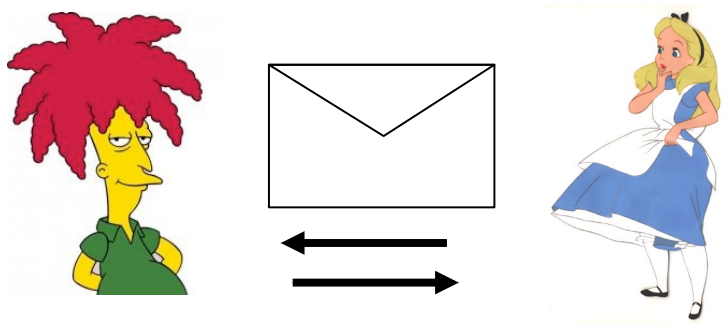


# How to use public key crypto?

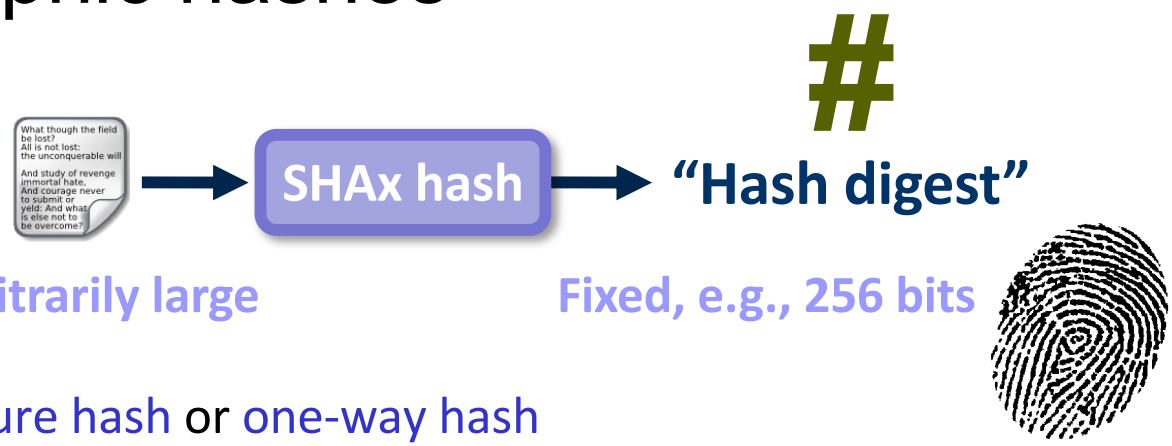
If B knows A's public key, that is sufficient for:

- A→B messages: authenticated but not secret.
- B→A messages: secret but not authenticated.

A and B can communicate securely in both directions if they each know the other's public key. (How? Later.)



# Cryptographic hashes

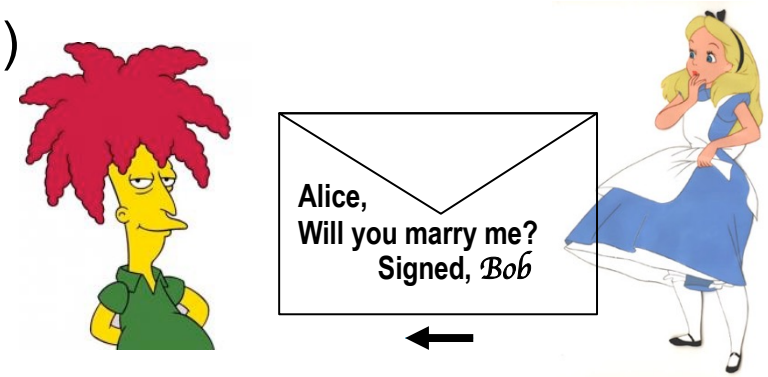



Also called a secure hash or one-way hash

- The hash function h takes a byte sequence M as input.
  - E.g., ~~SHA0, MD5~~, ~~SHA1~~, SHA2/**SHA-256**, SHA3, MD6
  - SHA-x: Secure Hashing Algorithm
- Very efficient, produces a small fixed-width result: h(M)
- Result is called a hash, checksum, fingerprint, digest

# Digital Signature

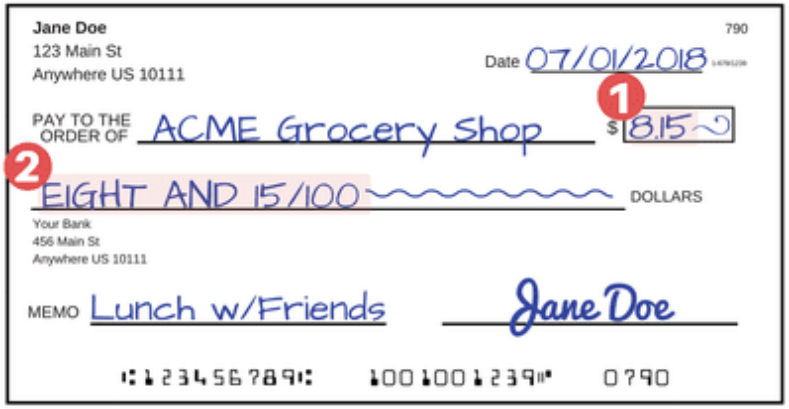
- Goal: authenticate + validate integrity of a message M
- Solution: take a hash digest h(M), encrypt it with B's private key, and append it to M as a digital signature
  - Unforgeable. "Proves" that B created M.
  - Certified. "Proves" M has not been tampered.
  - Non-repudiable. B cannot deny creating M.
  - (But not private.)



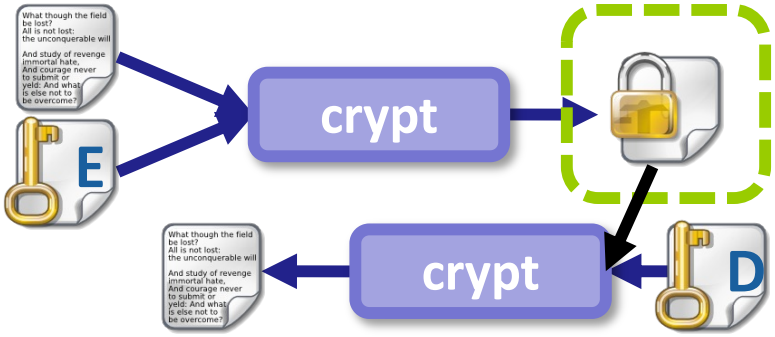
# A “key point”

A digital signature is “stronger” than a physical signature, because it is bound to the contents.

- Attacker cannot change the contents.
- Why? Because any change yields a different hash value, which invalidates the signature.
- And the attacker cannot correct it without the issuer’s private key.



# Relevant crypto: lightning summary




Generate a **keypair.**
Keep one key **private.**
The other key is **public**.

Use either key to **encrypt.**
Use the other to **decrypt**.

Use an open asymmetric (public-key) crypt function (e.g., **ECC**, **RSA**).
Combine with hashing (e.g., SHA256) to form **digital signatures**.

**Message** **Hash function** **fixed-width fingerprint** **"Computationally infeasible"**